HiAgent OpenAPI通用说明

本文档旨在为 HiAgent 平台的开发者提供一份清晰、统一的 API 通用说明，帮助您快速理解平台的接口规范并开始集成。

---

1. 接口分类

HiAgent 平台的 API 主要分为两大类，分别对应不同的使用场景和认证方式：

智能体 API (Agent API)

- **用途:** 主要用于与已创建的智能体进行对话交互。
- **特点:** 调用方式直接，专注于发布后的智能体对话以及周边能力。
- **访问方式:** https://{domain}/api/proxy/api/v1，比如 https://hia.volcenginepaas.com/api/proxy/api/v1

平台API (Platform API)

- **用途:** 涵盖平台级别的所有核心功能，如用户管理(IAM)、知识库(KBS)、应用管理(App)等。
- **特点:** 功能全面，提供对平台资源的程序化管理能力。
- **访问方式**：top-server-nodeport地址，一般为 `{Host IP}:30040`

2. 接口认证

根据接口分类的不同，平台提供两种相应的认证方案。

2.1 智能体API 认证：API Key

当调用智能体 API 时，您需要使用 `API Key` 进行认证。

- **获取方式:**
  a. 在智能体 **编排** 页面发布智能体配置。
  b. 在智能体 **发布** 页面，开启API配置开关并单击**渠道发布**，系统会为您生成API地址。
  c. `apikey` 通过点击 **API密钥** 来获取。



- **使用方法:** 将 `apikey` 放入 HTTP 请求的 `Apikey` 头中。

> **请求头格式** `Apikey: <YOUR_API_KEY>`

2.2 平台API认证：火山引擎鉴权

当调用平台接口时，需要采用标准的火山引擎 OpenAPI 鉴权方案。

- **获取方式:**
  a. 登录 HiAgent 平台。
  b. 通过 `点击头像 -> 账号中心 -> 密钥管理` 页面获取您的 `AccessKey` 和 `SecretAccessKey`。
- **使用方法:** 使用获取的密钥对您的请求进行签名。签名结果会附加在请求的 `Authorization` 头中。具体的签名算法请参考火山引擎的官方文档 https://www.volcengine.com/docs/6369/156029。

---

3. 通用请求与响应结构

3.1 智能体API

3.1.1 请求体结构

- **通用请求参数**

字段	位置	含义

Apikey	Header	智能体API key

其他参考各个接口说明。

3.1.2 通用返回内容

没有通用返回，具体返回内容请参考各个接口说明。

3.2 平台API

3.2.1 请求体结构

- **通用请求参数**

字段	位置	含义
Action	Query	代表请求的Action，例如：GetUserByID
Version	Query	代表请求的Version，例如：2022-01-01
X-Account-Id	Query	租户ID，如果是单租户，一般是1000000000
Region	Header	固定值：cn-north-1
AccessKey	Header	`AccessKey`
SecretKey	Header	`SecretAccessKey`

接口所需的其他参数放在body里面。

3.2.2 请求方式

所有请求均为POST请求。

3.2.3 返回结构

3.2.3.1 正常返回

代码块

```
{
    "ResponseMetadata": {
        "RequestId": "20201020172230010225O7", //从X-Top-Request-Id的header获取
        "Action": "{Action}",//从请求Action的query获取
        "Version": "{Version}",//从请求Version的query获取
```

```
    },
    "Result":"..."
}
```

- `ResponseMetadata` 是必须返回的， `Result` 是具体接口返回的内容，格式需满足标准 json。
- HTTP Code 为 200

3.2.3.2 异常返回

代码块

```
{
    "ResponseMetadata": {
        "RequestId": "20201020172230010225O7",
        "Action": "{Action}",
        "Version": "{Version}",
        "Error": {
            "Code": "InvalidActionOrVersion",
            "Message": "Could not find operation GetUserById for version 2018-01-012",
            "Data": {}
        }
    }
}
```

- 发生错误时不返回 `Result` 字段，并且 `ResponseMetadata` 中增加 `Error` 字段，包含错误的 `Code` 和 `Message` 等信息

---

4. 通用错误码

HTTP 状态码	错误类型	建议操作
401 Unauthorized	认证失败	请检查您的 `API Key` 或 `火山鉴权` 签名是否正确、有效。
400 Bad Request	参数错误	请检查请求体参数的格式、类型和完整性是否符合接口要求。
404 Not Found	资源不存在	请检查请求的 URL 或资源 ID 是否正确。
500 Internal Server Error	服务器内部错误	请稍后重试，如果问题持续存在，请联系技术支持。

> **注意:** 更详细的业务错误码，请参考具体接口的独立文档。

---

5. 请求示例

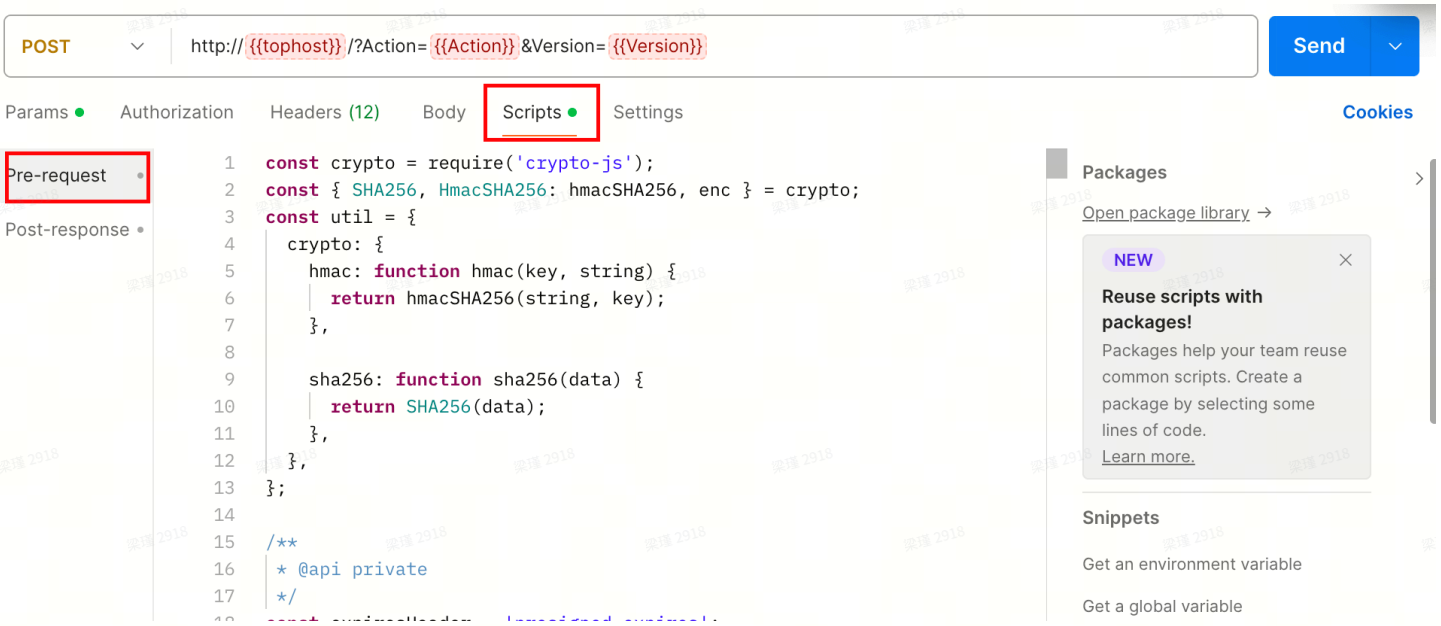
5.1 智能体API

参考各个接口提供的示例说明。

5.2 平台API

5.2.1 Postman

使用Postman访问平台接口需要先将签名脚本拷贝到Scripts->Pre-request（脚本具体内容如下，复制粘贴即可）



代码块

```
const crypto = require('crypto-js');
const { SHA256, HmacSHA256: hmacSHA256, enc } = crypto;
const util = {
  crypto: {
    hmac: function hmac(key, string) {
      return hmacSHA256(string, key);
    },

    sha256: function sha256(data) {
      return SHA256(data);
    },
```

```
  },
};

/**
 * @api private
 */
const expiresHeader = 'presigned-expires';

const unsignableHeaders = [
  'authorization',
  'content-type',
  'content-length',
  'user-agent',
  expiresHeader,
  'expect',
  'x-amzn-trace-id',
];

const uriEscape = (str) => {
  try {
    return encodeURIComponent(str)
      .replace(/[^A-Za-z0-9_.~\-%]+/g, escape)
      .replace(
        /[*]/g,
        (ch) => `%${ch.charCodeAt(0).toString(16).toUpperCase()}`
      );
  } catch (e) {
    return '';
  }
};

const queryParamsToString = (params, sort = true) =>
  (sort ? Object.keys(params).sort() : Object.keys(params))
    .map((key) => {
      const val = params[key];
      if (typeof val === 'undefined' || val === null) {
        return;
      }

      const escapedKey = uriEscape(key);
      if (!escapedKey) {
        return;
      }

      if (Array.isArray(val)) {
        return `${escapedKey}=${val
          .map(uriEscape)
```

```
          .sort()
          .join(`&${escapedKey}=`)}`;
      }

      return `${escapedKey}=${uriEscape(val)}`;
    })
    .filter((v) => v)
    .join('&');

/**
 * @api private
 */
class AWSSignersV4 {
  constructor(request, serviceName, options) {
    this.request = request;
    this.request.headers = request.headers || {};
    this.serviceName = serviceName;
    options = options || {};
    this.signatureCache =
      typeof options.signatureCache === 'boolean'
        ? options.signatureCache
        : true;
    this.operation = options.operation;
    this.signatureVersion = options.signatureVersion;
    this.constant = options.isVolcengine
      ? {
          algorithm: 'HMAC-SHA256',
          v4Identifier: 'request',
          dateHeader: 'X-Date',
          tokenHeader: 'x-security-token',
          contentSha256Header: 'X-Content-Sha256',
          kDatePrefix: '',
        }
      : {
          algorithm: 'AWS4-HMAC-SHA256',
          v4Identifier: 'aws4_request',
          dateHeader: 'X-Amz-Date',
          tokenHeader: 'x-amz-security-token',
          contentSha256Header: 'X-Amz-Content-Sha256',
          kDatePrefix: 'AWS4',
        };
    this.bodySha256 = options.bodySha256;
  }

  addAuthorization(credentials, date) {
    const datetime = this.iso8601(date).replace(/[:\-]|\.\d{3}/g, '');
    this.addHeaders(credentials, datetime);
```

```
106       this.request.headers['Authorization'] = this.authorization(
107         credentials,
108         datetime
109       );
110     }
111
112     addHeaders(credentials, datetime) {
113       this.request.headers[this.constant.dateHeader] = datetime;
114       if (credentials.sessionToken) {
115         this.request.headers[this.constant.tokenHeader] =
116           credentials.sessionToken;
117       }
118       if (this.request.body) {
119         let body = this.request.body;
120         if (typeof body !== 'string') {
121           if (body instanceof URLSearchParams) {
122             body = body.toString();
123           } else {
124             body = JSON.stringify(body);
125           }
126         }
127         this.request.headers[this.constant.contentSha256Header] =
128           this.bodySha256 || util.crypto.sha256(body).toString();
129       }
130     }
131
132     authorization(credentials, datetime) {
133       const parts = [];
134       const credString = this.credentialString(datetime);
135       parts.push(
136         `${this.constant.algorithm}
    Credential=${credentials.accessKeyId}/${credString}`
137       );
138       parts.push(`SignedHeaders=${this.signedHeaders()}`);
139       const signature = this.signature(credentials, datetime);
140       console.log('signature is', signature.toString(enc.Hex));
141       parts.push(`Signature=${signature}`);
142       return parts.join(', ');
143     }
144
145     signature(credentials, datetime) {
146       const signingKey = this.getSigningKey(
147         credentials,
148         datetime.substr(0, 8),
149         this.request.region,
150         this.serviceName,
151         this.signatureCache
```

```
    );
    const signStr = this.stringToSign(datetime);
    console.log('signStr is', signStr);
    console.log('signingKey is', signingKey.toString(enc.Hex));
    return util.crypto.hmac(signingKey, signStr, 'hex');
  }

  stringToSign(datetime) {
    const parts = [];
    parts.push(this.constant.algorithm);
    parts.push(datetime);
    parts.push(this.credentialString(datetime));
    const canonicalString = this.canonicalString();
    console.log('canonicalString is', canonicalString);
    parts.push(this.hexEncodedHash(canonicalString));

    return parts.join('\n');
  }

  canonicalString() {
    const parts = [],
      pathname = this.request.pathname || '/';

    parts.push(this.request.method.toUpperCase());
    parts.push(pathname);
    parts.push(queryParamsToString(this.request.params) || '');
    parts.push(`${this.canonicalHeaders()}\n`);
    parts.push(this.signedHeaders());
    parts.push(this.hexEncodedBodyHash());
    return parts.join('\n');
  }

  canonicalHeaders() {
    const headers = [];
    Object.keys(this.request.headers).forEach((key) => {
      headers.push([key, this.request.headers[key]]);
    });
    headers.sort((a, b) => (a[0].toLowerCase() < b[0].toLowerCase() ? -1 : 1));
    const parts = [];
    headers.forEach((item) => {
      const key = item[0].toLowerCase();
      if (this.isSignableHeader(key)) {
        const value = item[1];
        if (
          typeof value === 'undefined' ||
          value === null ||
          typeof value.toString !== 'function'
```

```
        ) {
          throw new Error(`Header ${key} contains invalid value`);
        }
        parts.push(`${key}:${this.canonicalHeaderValues(value.toString())}`);
      }
    });
    return parts.join('\n');
  }

  canonicalHeaderValues(values) {
    return values.replace(/\s+/g, ' ').replace(/^\s+|\s+$/g, '');
  }

  signedHeaders() {
    const keys = [];
    Object.keys(this.request.headers).forEach((key) => {
      key = key.toLowerCase();
      if (this.isSignableHeader(key)) {
        keys.push(key);
      }
    });
    return keys.sort().join(';');
  }

  credentialString(datetime) {
    return this.createScope(
      datetime.substr(0, 8),
      this.request.region,
      this.serviceName
    );
  }

  hexEncodedHash(string) {
    return util.crypto.sha256(string);
  }

  hexEncodedBodyHash() {
    if (this.request.headers[this.constant.contentSha256Header]) {
      return this.request.headers[this.constant.contentSha256Header];
    }

    if (this.request.body) {
      return this.hexEncodedHash(queryParamsToString(this.request.body));
    }
    return this.hexEncodedHash('');
  }

```

```
246     isSignableHeader(key) {
247       if (key.toLowerCase().indexOf('x-amz-') === 0) {
248         return true;
249       }
250       return unsignableHeaders.indexOf(key) < 0;
251     }
252
253     iso8601(date) {
254       if (date === undefined) {
255         date = new Date();
256       }
257       return date.toISOString().replace(/\.\d{3}Z$/, 'Z');
258     }
259
260     getSigningKey(credentials, date, region, service) {
261       // const credsIdentifier = crypto.hmac(credentials.secretAccessKey,
    credentials.accessKeyId, 'base64');
262       // const cacheKey = [credsIdentifier, date, region, service].join('_');
263
264       const kDate = util.crypto.hmac(
265         `${this.constant.kDatePrefix}${credentials.secretAccessKey}`,
266         date
267       );
268       // debugger;
269       const kRegion = util.crypto.hmac(kDate, region);
270       const kService = util.crypto.hmac(kRegion, service);
271
272       const signingKey = util.crypto.hmac(kService, this.constant.v4Identifier);
273
274       return signingKey;
275     }
276
277     createScope(date, region, serviceName) {
278       return [
279         date.substr(0, 8),
280         region,
281         serviceName,
282         this.constant.v4Identifier,
283       ].join('/');
284     }
285   }
286
287   /**
288    * 获取值 支持变量语法
289    * @param {string} value
290    * @returns
291    */
```

```
function getValue(value = '') {
  const variableKeyMatch = value.match(/^\{\{(.+?)\}\}$/);
  const variableKey = variableKeyMatch && variableKeyMatch[1];
  if (variableKey) {
    return pm.environment.get(variableKey) || value;
  } else {
    return value;
  }
}

/**
 * 将postman的kv list转换为map 并过滤disabled参数
 * @param {*} list
 * @returns
 */
function getMap(list) {
  const map = {};
  list.forEach((item) => {
    if (!item.disabled) {
      const value = typeof item.value === 'string' ? getValue(item.value) : '';
      // 重复query处理
      if (map[item.key]) {
        map[item.key] = Array.isArray(map[item.key])
          ? [...map[item.key], value]
          : [map[item.key], value];
      } else map[item.key] = value;
    }
  });
  return map;
}
// 获取转换后的headers和params
const headers = getMap(pm.request.headers.all());
const params = getMap(pm.request.url.query.all());

// 从headers中解析出签名所需参数
const {
  AccessKey,
  SecretKey,
  ServiceName,
  Region = 'cn-north-1',
  isVolcEngine = 'true',
  'X-Security-Token': sessionToken,
} = headers;

// 经过处理的header 不再透传
const handleKeys = [
  'AccessKey',
```

```
    'SecretKey',
    'ServiceName',
    'Region',
    'isVolcEngine',
    'X-Security-Token',
  ];
  handleKeys.forEach((key) => {
    delete headers[key];
    pm.request.headers.remove(key);
  });

  // 构造签名对象
  const signObj = {
    region: Region,
    headers: {}, // 默认不签名用户传递的headers 防止tlb改header导致的签名错误 本代码用于开发自测 这部分安全性可以忽略
    params,
    method: pm.request.method,
    pathname: '/' + (pm.request.url.path ? pm.request.url.path.join('/') : ''),
  };

  // 串化各种情况下的body
  if (pm.request.body) {
    switch (pm.request.body.mode) {
      case 'raw':
        signObj.body = pm.request.body.raw;
        break;
      case 'urlencoded':
        const bodys = getMap(pm.request.body.urlencoded.all());
        signObj.body = queryParamsToString(bodys, false);
        break;
      case 'formdata': {
        const enableBodys = pm.request.body.formdata.filter(
          (item) => !item.disabled
        );
        if (enableBodys.length === 0) {
          break;
        }
        // 自己生成boundary
        const boundary = `--------------------------${Date.now()}`;
        pm.request.headers.add({
          key: 'Content-Type',
          value: `multipart/form-data; boundary=${boundary}`,
        });
        signObj.body = `--${boundary}\r\n${enableBodys
          .map(
            (item) =>
```

```
            `Content-Disposition: form-data; name="${
              item.key
            }"\r\n\r\n${getValue(item.value)}\r\n`
        )
        .join(`--${boundary}\r\n`)}--${boundary}--\r\n`;
      break;
    }
    default:
      break;
  }
}

// 执行签名
const signer = new AWSSignersV4(signObj, ServiceName, {
  isVolcengine: isVolcEngine !== 'false',
});
signer.addAuthorization({
  accessKeyId: AccessKey,
  secretAccessKey: SecretKey,
  sessionToken,
});

// 将签名后的headers注入到请求的headers中
Object.keys(signer.request.headers).forEach((key) => {
  pm.request.headers.add({
    key,
    value: signer.request.headers[key],
  });
});

// 对query做encode处理 同时通过queryParamsToString保证签名时的querystring与发送出请求的一致
if (pm.request.url.query) {
  pm.request.url.query = queryParamsToString(params);
}

// 构造curl命令 用于自行debug
try {
  function getCurlBodyString(body) {
    if (!body) return '';
    switch (body.mode) {
      case 'raw':
        return `--header 'Content-Type: application/json' \\
--data-raw '${body.raw}'`;
      case 'urlencoded':
        const bodys = getMap(body.urlencoded.all());
        const bodyString = queryParamsToString(bodys, false);
```

```
          return `--header 'Content-Type: application/x-www-form-urlencoded' \\
${bodyString
  .split('&')
  .map((v) => `--data-urlencode '${v}'`)
  .join(' \\\n')}`;
        case 'formdata': {
          // formdata怎么curl 还没想明白
          throw new Error('can not generate curl of formdata');
        }
        default:
          break;
      }
    }
    const headerString = pm.request.headers
      .toString()
      .split('\n')
      .map((line) => `--header '${line}'`)
      .join(' \\\n');
    const bodyString = getCurlBodyString(pm.request.body);
    const url = `${pm.request.url.protocol}://${pm.request.url.host.join('.')}${
      pm.request.path ? pm.request.path.join('/') : '/'
    }?${pm.request.url.query}`;
    // 打印出curl请求
    console.log(
      [
        `curl --location --request ${pm.request.method} '${url}'`,
        headerString,
        bodyString,
      ]
        .filter((v) => !!v)
        .join(' \\\n')
    );
  } catch (e) {
    console.log(e);
  }
```

具体Postman示例参考各个接口提供的文档。

5.2.2 Java代码示例

推荐通过Maven依赖使用火山引擎SDK for Java：详情见：火山引擎SDK for Java

代码块

```
<dependency>
    <groupId>org.apache.httpcomponents</groupId>
    <artifactId>httpclient</artifactId>
```

```
    <version>4.5.8</version>
</dependency>
<dependency>
    <groupId>com.volcengine</groupId>
    <artifactId>volc-sdk-java</artifactId>
    <version>1.0.49</version>
</dependency>
```

代码块

```
package com.example;

import com.volcengine.auth.ISignerV4;
import com.volcengine.auth.impl.SignerV4Impl;
import com.volcengine.model.Credentials;
import com.volcengine.service.SignableRequest;
import org.apache.http.HttpEntity;
import org.apache.http.HttpResponse;
import org.apache.http.client.config.RequestConfig;
import org.apache.http.client.utils.URIBuilder;
import org.apache.http.entity.ByteArrayEntity;
import org.apache.http.entity.ContentType;
import org.apache.http.client.HttpClient;
import org.apache.http.impl.client.HttpClientBuilder;
import org.apache.http.util.Asserts;
import org.apache.http.util.EntityUtils;

public class RequestHandler {

    private ISignerV4 singer;
    private Credentials credentials = new Credentials();
    private HttpClient httpClient;
    private String url;

    public void GenRequest(String ak, String sk, String host, String action,
String version) {
        singer = new SignerV4Impl();
        RequestConfig requestConfig =
RequestConfig.custom().setConnectionRequestTimeout(1000)
                .setSocketTimeout(3000).setConnectTimeout(1000).build();
        httpClient =
HttpClientBuilder.create().setDefaultRequestConfig(requestConfig).build();

        credentials.setAccessKeyID(ak);
```

```
32          credentials.setSecretAccessKey(sk);
33          credentials.setRegion("cn-north-1");
34          credentials.setService("app");
35
36          url = host + "/?Action=" + action + "&Version=" + version;
37      }
38
39      // 请求数据
40      public void sendByteData(byte[] data) throws Exception {
41          send(data, ContentType.APPLICATION_JSON);
42      }
43
44      private void send(byte[] data, ContentType contentType) throws Exception {
45          SignableRequest signableRequest;
46          signableRequest = new SignableRequest();
47          signableRequest.setMethod("POST");
48          URIBuilder uriBuilder = new URIBuilder(url);
49
50          signableRequest.setHeader("Content-Type", contentType.getMimeType());
51
52          signableRequest.setEntity(new ByteArrayEntity(data, contentType));
53          signableRequest.setUriBuilder(uriBuilder);
54          signableRequest.setURI(uriBuilder.build());
55
56          singer.sign(signableRequest, credentials);
57
58          Asserts.notNull(signableRequest, "signableRequest can not null");
59
60          HttpResponse response = httpClient.execute(signableRequest);
61
62          if (response.getStatusLine().getStatusCode() != 200) {
63              throw new Exception("http status code error");
64          }
65
66          HttpEntity respEntity = response.getEntity();
67          byte[] content = EntityUtils.toByteArray(respEntity);
68
69          String answer = new String(content, "UTF-8");
70          System.out.println(answer);
71      }
72
73      public static void main(String[] args) throws Exception {
74          RequestHandler h = new RequestHandler();
75
76          // TOP Host获取可以咨询交付同学
77          h.GenRequest("${AK}", "${SK}", "${TOP Host}", "${Action}",
    "${Version}");
```

```
78
79          h.sendByteData("{\"TenantID\":\"1000000000\"}".getBytes());
80      }
81  }
```

更多代码示例参考：HiAgent平台API调用示例（适用于V1.4.x及以后版本）